Memory cache pentru aplicații web

Tabele in-memory + bază de date relațională — implementare distribuită în Rust și evaluarea performanțelor

Cristi Miloiu

8 noiembrie 2026

Proiect individual — aplicații paralele și distribuite

Problema

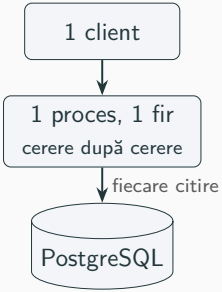
- Magazin online: 100k produse, 500k recenzii, 200 de categorii în PostgreSQL
- Trafic dominat de citiri (90–99%), popularitate Zipf
- Fiecare citire = join + agregare în DB
- Întrebarea: *debit maxim, latență minimă, date corecte, memorie limitată, mai multe mașini?*

$$T_{\text{mediu}} = T_{\text{cache}} + (1 - h)\, T_{\text{DB}}$$

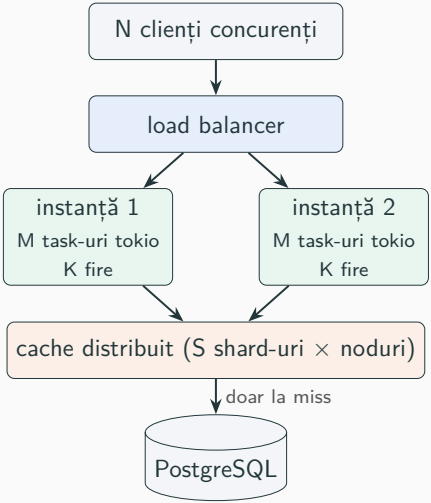
Rata de hit h domină: de la 90% la 99% = de 10× mai puține interogări DB.

Secvențial vs. paralel și distribuit

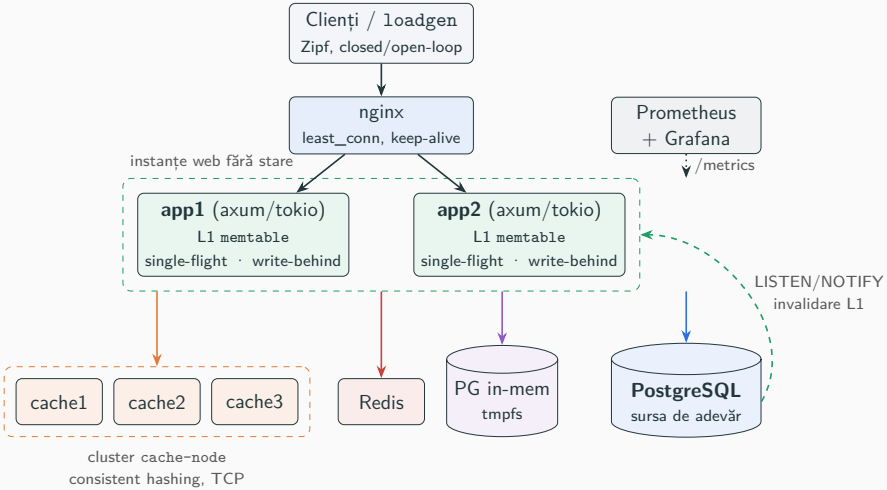
(a) Secvențial



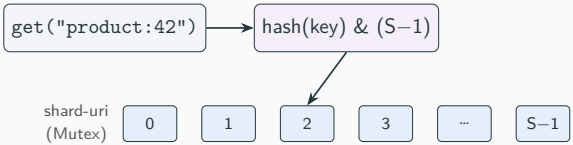
(b) Paralel și distribuit



Arhitectura



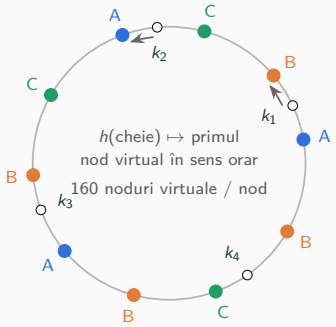
Motorul `memtable`



LRU: la acces, nodul se mută în cap · FIFO: nu se mută · CLOCK: bit de referință, „a doua șansă” la coadă

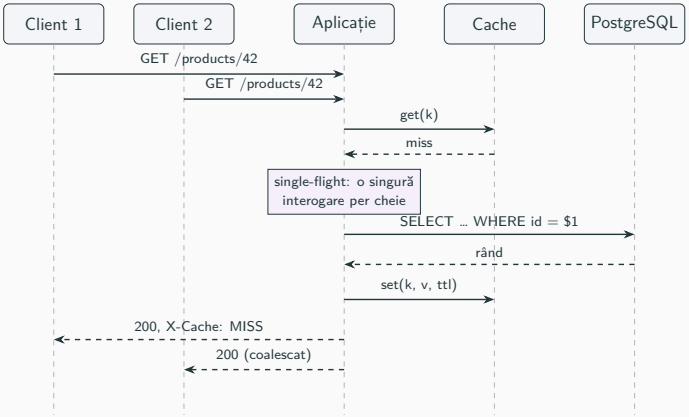
Shard-uri cu mutex propriu · slab + listă intrusivă · O(1) · LRU/FIFO/CLOCK · TTL · tag-uri

Cache distribuit și citirea cu single-flight



consistent hashing, 160 noduri virtuale

protocol TCP propriu, pipelining



Strategii de scriere și coerență

Invalidate (cache-aside)

1. UPDATE în DB
2. DEL cheie din cache
3. următoarea citire: miss → DB

\+ simplu, corect
− miss după fiecare scriere

Write-through

1. UPDATE în DB
2. SET valoarea nouă în cache
3. citirile rămân hit

\+ hit ratio păstrat
− latență de scriere mai mare

Write-behind

1. SET în cache; răspuns 202
2. coadă → lot ($\leq$1024)
3. un UPDATE … unnest()

\+ scrieri rapide, grupate
− consistență eventuală, risc la crash

- Coerența L1 între instanțe: trigger PostgreSQL → `NOTIFY` → `LISTEN` în fiecare instanță
- Test end-to-end: 18/18 combinații backend × strategie coerente

Șase backend-uri, comutabile la runtime

`none`	fără cache — baseline
`local`	`memtable` în proces (L1)
`cluster`	3 noduri `cache-node`, consistent hashing
`tiered`	L1 local (TTL 5 s) + L2 cluster — near-cache
`redis`	Redis 8 — baseline industrial
`pgmem`	tabelă `UNLOGGED` în PostgreSQL pe `tmpfs` — „tabelă in-memory" relațională

UI de control · Swagger/OpenAPI · Prometheus + Grafana · Docker Compose (11 servicii)

Optimizări ghidate de profilare

1. Profilare: PostgreSQL la 410% CPU, aplicațiile sub 20%
2. Fără paralelism intra-query + indecși acoperitori ⇒ clasament 7,6 ms → 1,5 ms
3. Calea fierbinte: mimalloc, metrici pre-rezolvate, fără tracing per cerere
4. `NOTIFY` serializa toate commit-urile (`Lock:object`) ⇒ notificare doar când există L1
5. Agregate materializate + index ⇒ clasament 0,09 ms; PostgreSQL de la 410% la ≈60–100% CPU

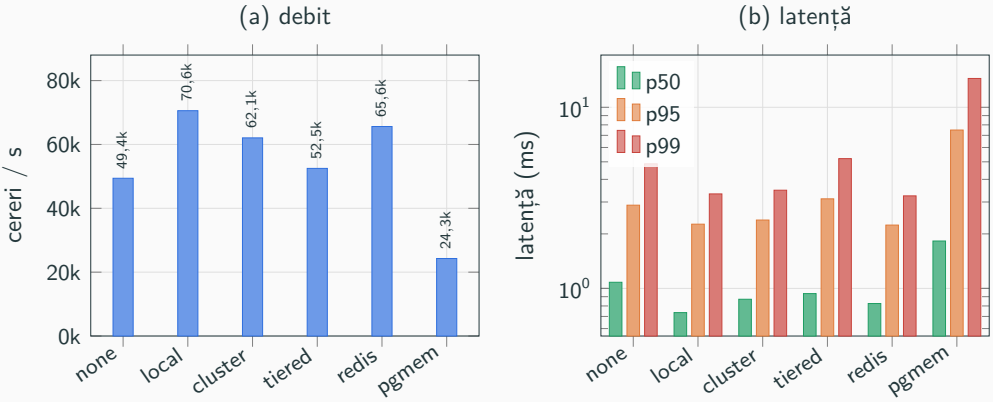
	înainte	după
fără cache (cereri/s)	8.190	50.356
cache local (cereri/s)	12.850	67.933
cache local p99 (ms)	62,1	3,5

Rezultate experimentale

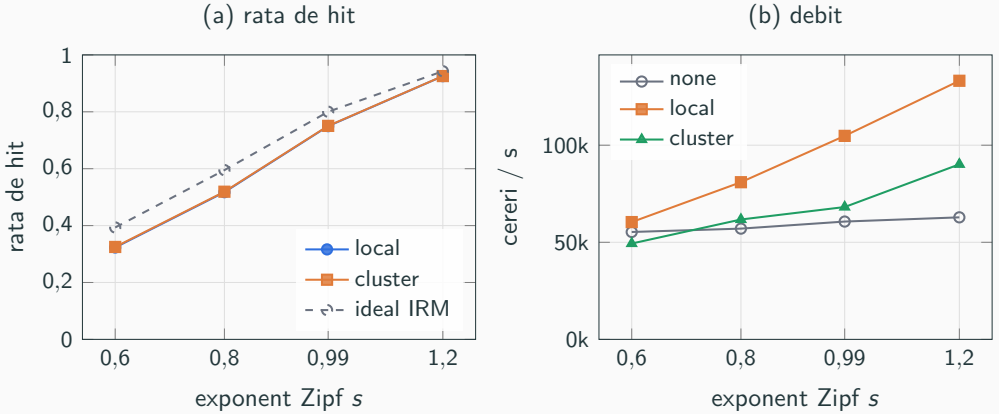
Metodologie

- 5 vCPU (Docker Desktop), toate componentele pe aceeași mașină
- `loadgen` propriu: closed-loop, Zipf, histograme HDR, 5 s încălzire + 15 s măsurare
- Implicit: 64 clienți, 95% citiri (10% dintre ele clasamente), Zipf $s = 0{,}99$
- 8 experimente HTTP + 2 benchmark-uri offline; totul scriptat (`make bench`)

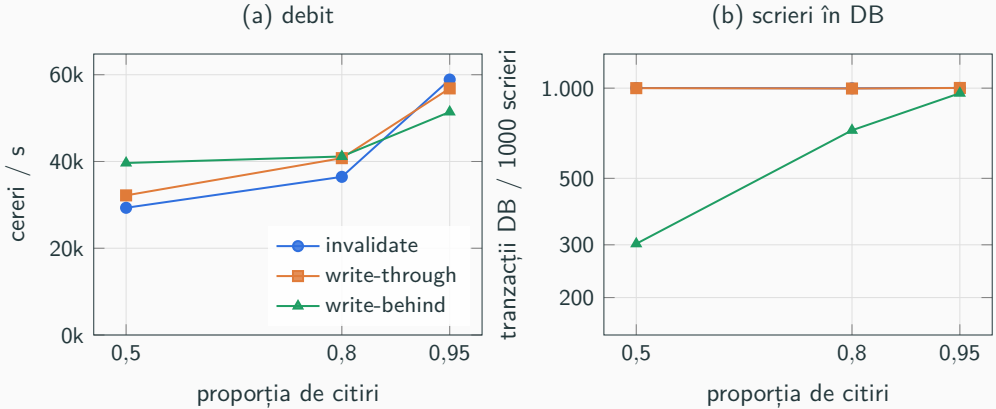
E1 — comparația backend-urilor



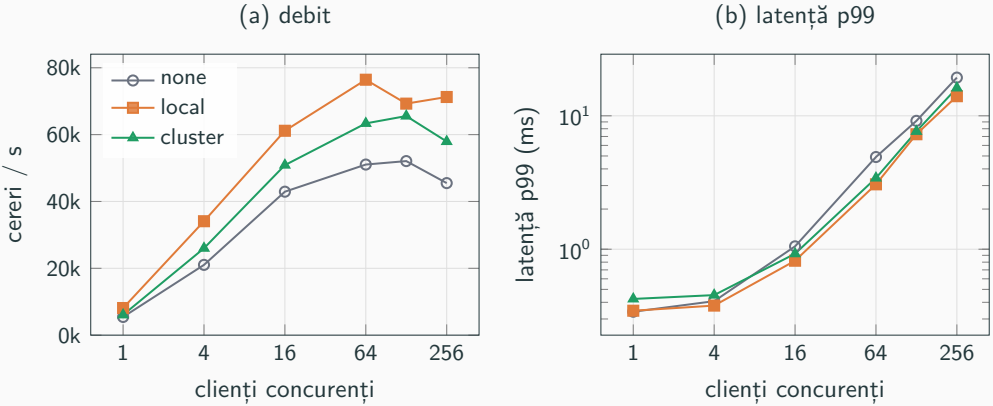
E2 — asimetria Zipf (cache = 10% din chei)



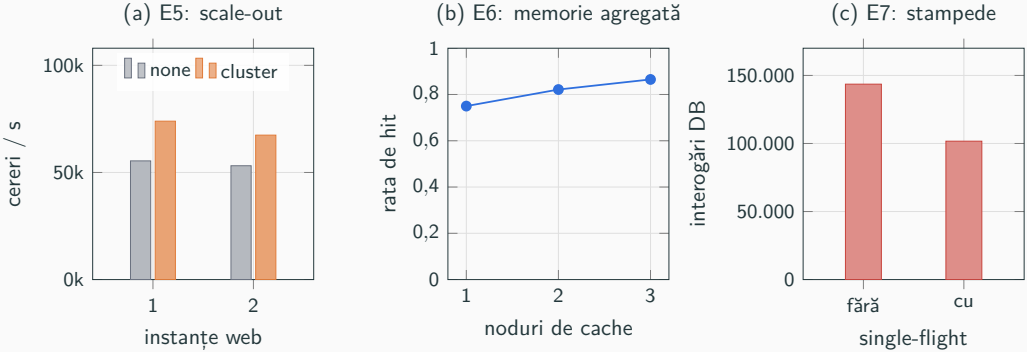
E3 — strategii de scriere



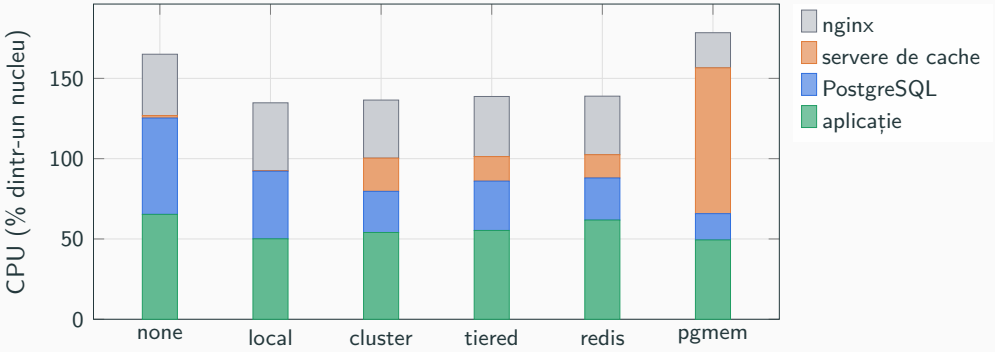
E4 — scalabilitate cu numărul de clienți



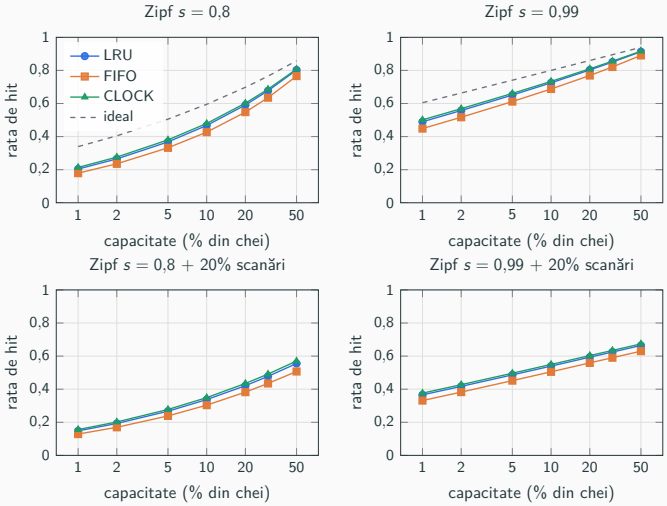
E5–E7 — scale-out, memorie agregată, stampede



E8 — costul CPU la debit fix

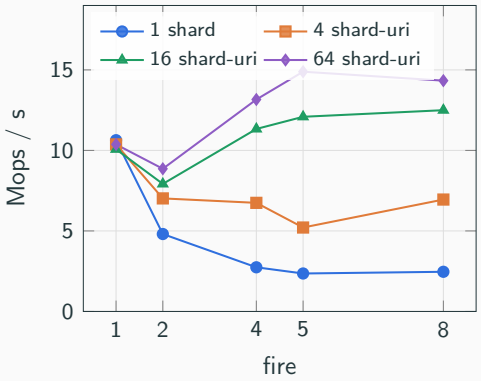


Politici de evacuare (simulare)

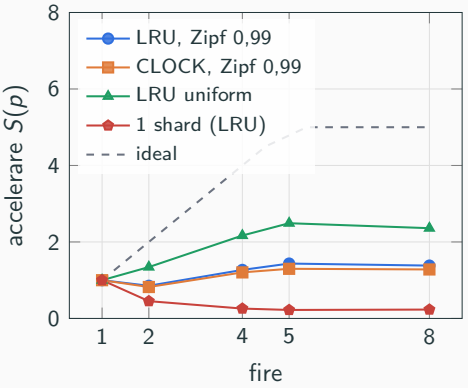


Scalabilitatea motorului: fire × shard-uri

(a) debit (LRU, Zipf 0,99)



(b) accelerare, 64 shard-uri



Concluzii

- Întâi baza de date, apoi cache-ul: 12.850 → 67.933 cereri/s, p99 62,1 → 3,5 ms
- Cache local: 1,43× debit față de DB optimizat, −27% CPU per cerere
- Cache partajat (cluster, Redis): hit mai mare, coerență mai ieftină; prag de rentabilitate (E2)
- Tabela in-memory în SGBD (`pgmem`): mai lentă decât DB-ul pentru date deja în RAM
- Write-behind: debit maxim, de câteva ori mai puține tranzacții, consistență eventuală
- Sharding: de la colaps (1 shard) la scalare; cheile fierbinți limitează accelerarea
- Single-flight: −29% interogări la pornirea la rece; coerență 18/18

Întrebări?

`make up` → `http://localhost:8080`